

DocAI System Architecture and Implementation Report

Version: 1.0 **Date:** 2025-12-04 **Author:** System Analysis by Claude

Table of Contents

- 1. System Profile and Overview
- 2. Data Flow and Workflow Analysis
- 3. Module-Level Architecture
- 4. Class-Level Design
- 5. Intra-Class Function Analysis
- 6. Data Model Explanation
- 7. Summary

1. System Profile and Overview

1.1 What is DocAI?

DocAI is a local **RAG (Retrieval-Augmented Generation)** system. Think of it as your personal librarian who not only knows where every book is stored but can also read, understand, and answer questions about them.

Analogy: Imagine a massive library where:

- You upload documents (books) to the library
- The librarian (DocAI) reads and indexes every page
- When you ask a question, the librarian finds the most relevant passages
- Then, an AI assistant reads those passages and formulates an answer

1.2 Dual RAG Architecture

DocAI implements **two parallel systems** that operate independently:

DocAI System	
File-Based RAG (Legacy System)	Skill-Based RAG (New Architecture)
- Per-file indexing - Milvus vector store 5-Phase OPMP Pipeline - SSE streaming - Multi-file selection	- Curated knowledge domains - FAISS vector store - Simplified query pipeline - JSON response - Multi-skill selection

File-Based RAG = Individual PDF files, searchable separately **Skill-Based RAG** = Curated "Skills" (knowledge domains) combining multiple PDFs

1.3 Technology Stack

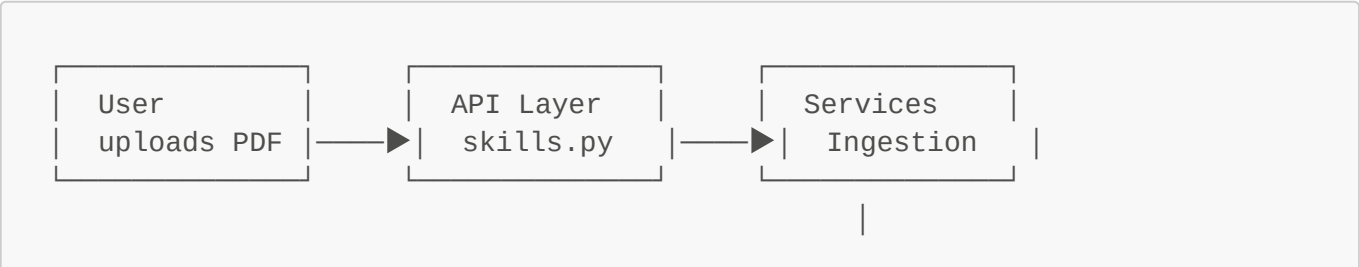
Component	Technology	Purpose
Backend	Python 3.11, FastAPI	API server, async processing
Vector DB (File)	Milvus	High-performance vector search
Vector DB (Skill)	FAISS	Lightweight, physically isolated indices
Metadata DB	SQLite	Skill metadata, relationships
Embeddings	BAAI/bge-m3	1024-dimensional text embeddings
LLM	Ollama (local) / OpenAI	Response generation
Frontend	Vanilla JS + Jinja2	User interface

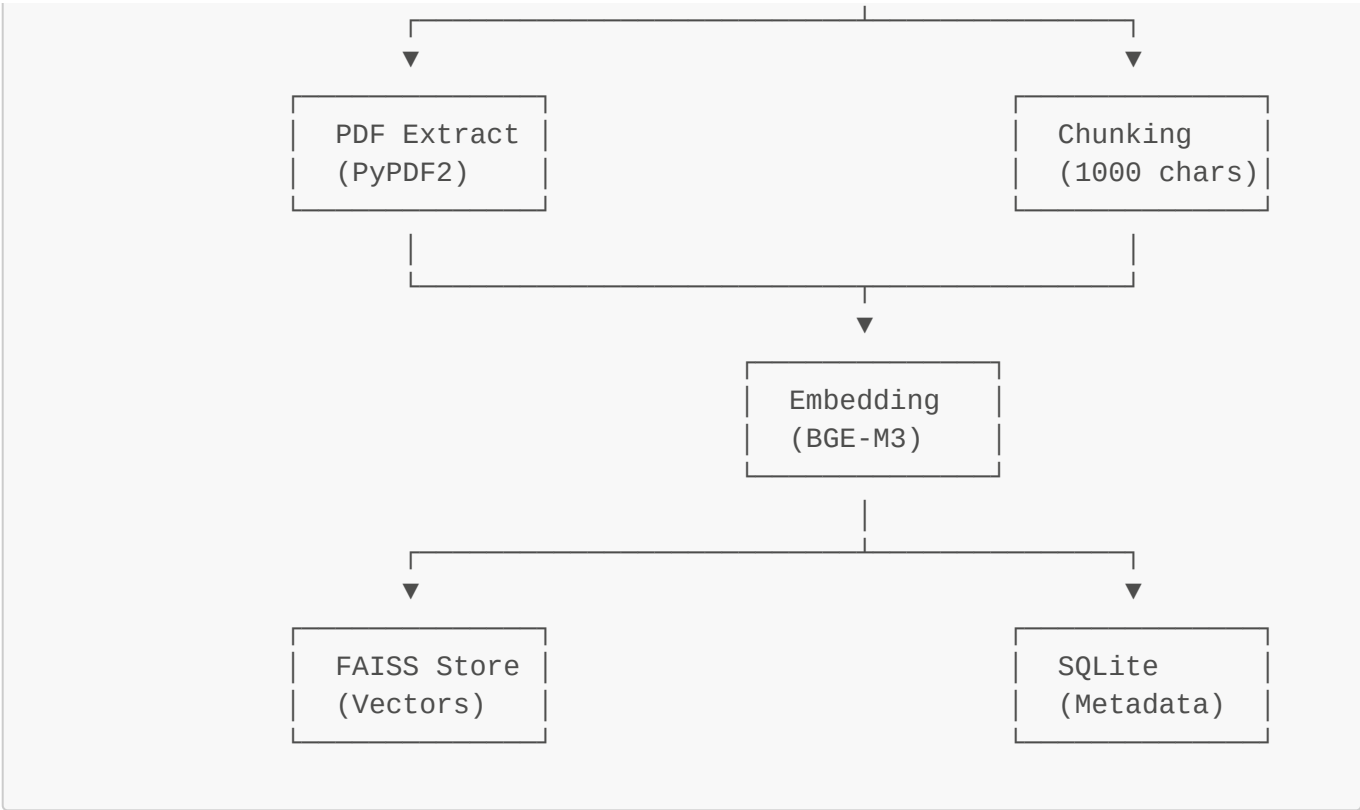
1.4 Directory Structure

```
DocAI/
├── app/
│   ├── api/v1/endpoints/      # FastAPI route handlers
│   ├── models/                # Data models (TypedDict + Pydantic)
│   ├── Providers/             # Infrastructure layer (DB, Vector, LLM)
│   ├── Services/              # Business logic (File-based)
│   └── SkillServices/         # Business logic (Skill-based)
├── data/
│   ├── faiss_indices/         # Vector storage
│   │   ├── files/             # File-based embeddings
│   │   └── skills/            # Skill-based embeddings
│   ├── skill_metadata.db      # SQLite database
│   └── docai.db               # Legacy file metadata
├── template/                  # HTML templates
└── scripts/                   # Utility scripts
```

2. Data Flow and Workflow Analysis

2.1 Document Upload Flow (Skill-Based)

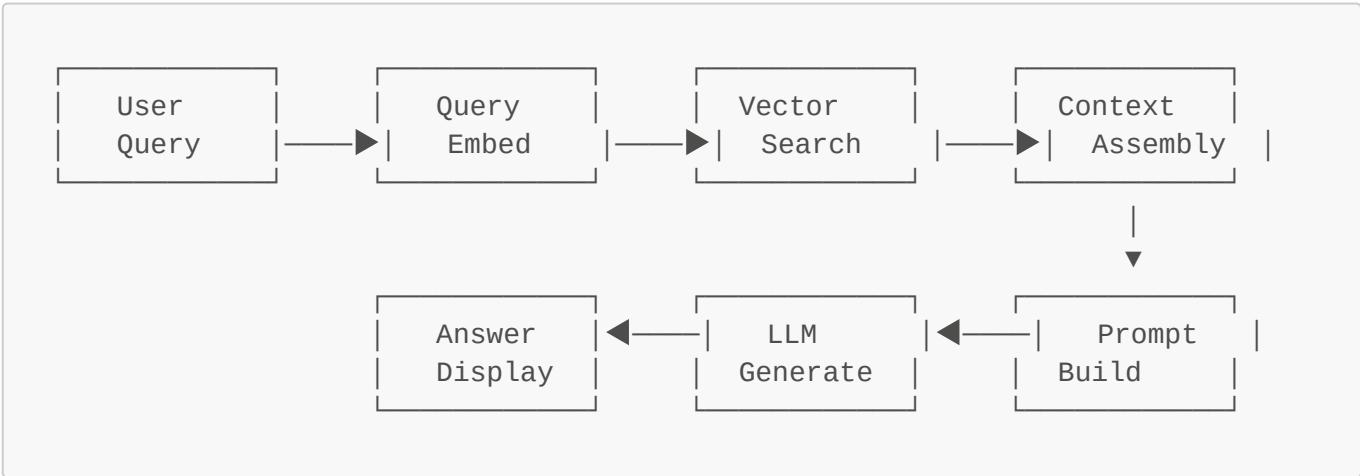




Step-by-step:

- 1. **Upload:** User submits PDF via `/api/v1/skills/{skill_name}/upload-source`
- 2. **Save:** PDF saved to `data/pdf_sources/{skill_name}/`
- 3. **Extract:** PyPDF2 extracts text from each page
- 4. **Chunk:** Text split into ~1000 character chunks with overlap
- 5. **Embed:** BGE-M3 generates 1024-dim vectors for each chunk
- 6. **Store Vectors:** FAISS index saved to `data/faiss_indices/skills/{skill_id}/`
- 7. **Store Metadata:** SQLite records skill info, chunk counts

2.2 Query Flow (RAG Pipeline)



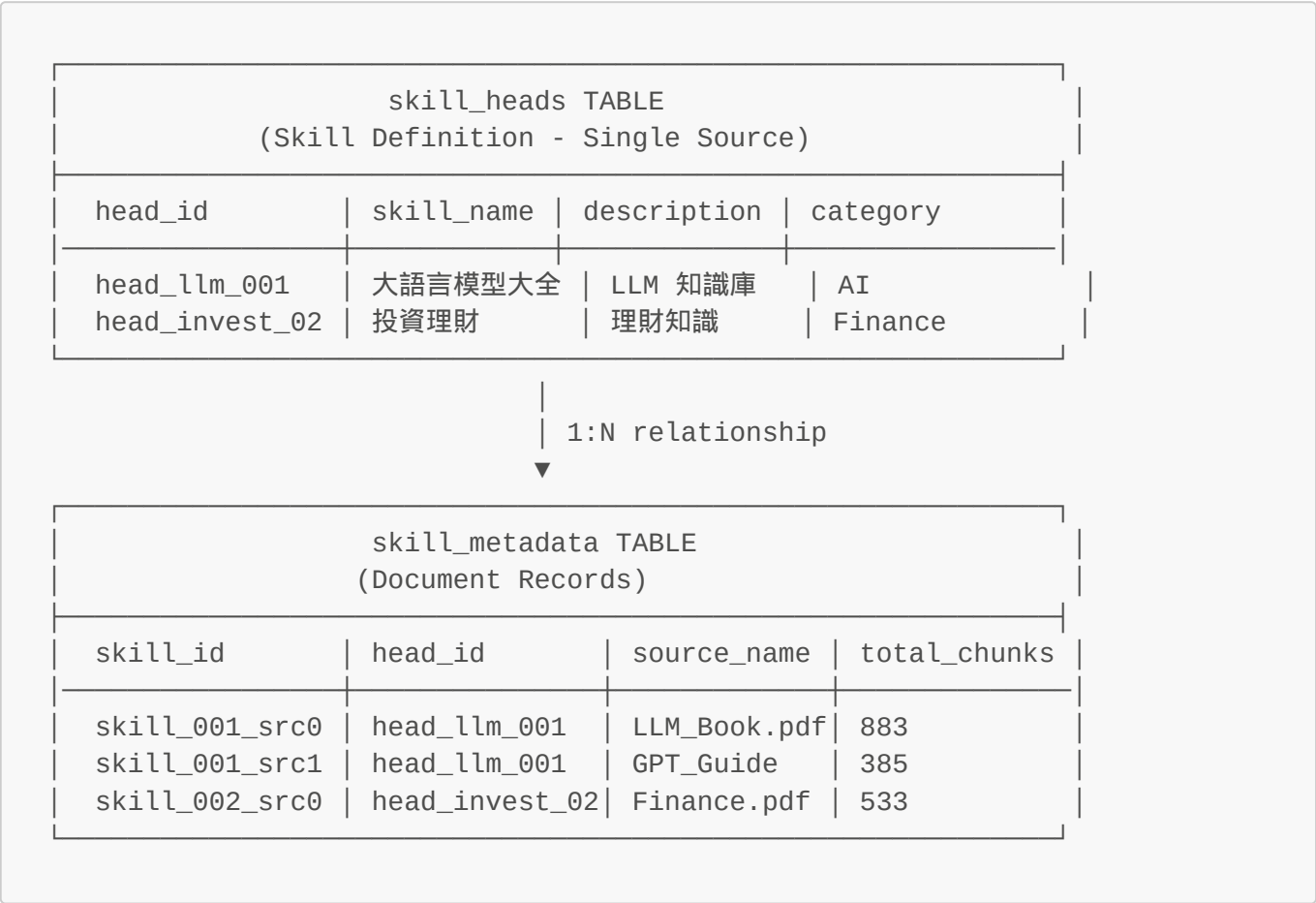
Step-by-step:

- 1. **User Query:** "What is machine learning?"
- 2. **Query Embedding:** BGE-M3 converts query to 1024-dim vector
- 3. **Vector Search:** FAISS finds top-K similar chunks (default K=10)

4. **Context Assembly:** Retrieved chunks formatted as context
5. **Prompt Build:** System prompt + context + user query combined
6. **LLM Generation:** Ollama/OpenAI generates response
7. **Response:** Answer returned with source citations

2.3 skill_heads Architecture (Single Source of Truth)

The new `skill_heads` table solved the **Dual Source of Truth** anti-pattern:

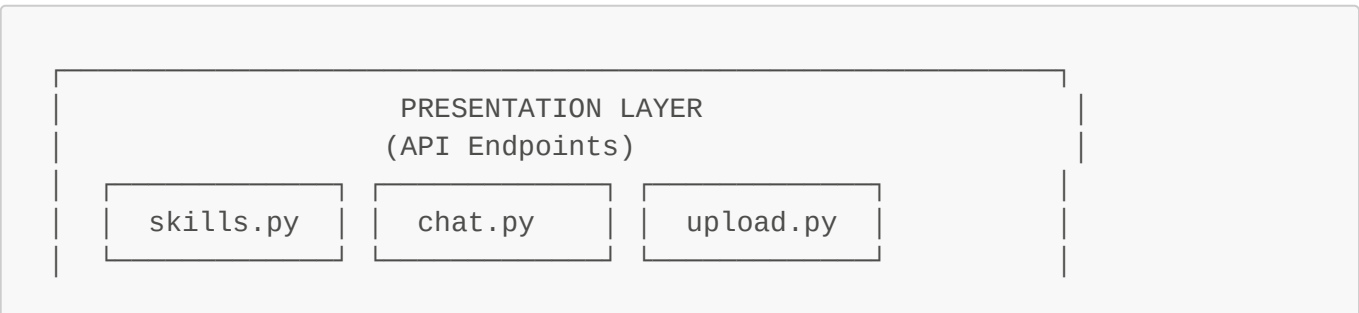


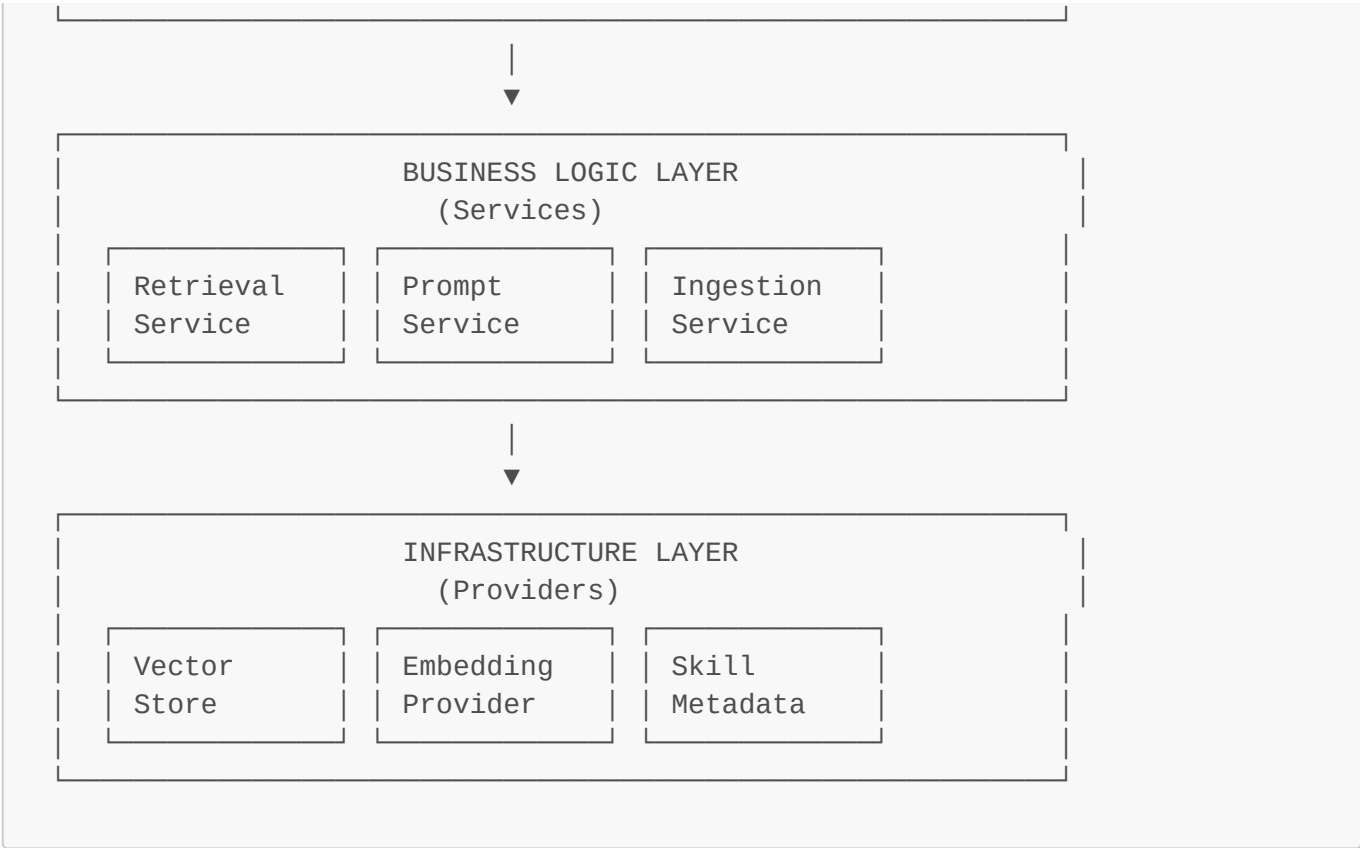
Why this design?

- **Before:** skill_config.json + skill_metadata.db = Sync issues
- **After:** skill_heads table = Single Source of Truth
- **Benefit:** Creating a new Skill automatically creates a visible header in UI

3. Module-Level Architecture

3.1 Layer Architecture





3.2 Module Responsibilities

Module	Location	Responsibility
API Endpoints	app/api/v1/endpoints/	HTTP request handling, validation
Services (File)	app/Services/	File-based RAG business logic
Services (Skill)	app/SkillServices/	Skill-based RAG business logic
Providers	app/Providers/	Infrastructure abstraction
Models	app/models/	Data structure definitions

3.3 Provider Modules Detail

```
Providers/
├── embedding_provider/      # Text → Vector conversion
│   └── client.py           # EmbeddingProvider class
├── vector_store_provider/  # FAISS/Milvus operations
│   └── client.py           # VectorStoreProvider class
├── skill_metadata_provider/ # SQLite skill operations
│   └── client.py           # SkillMetadataProvider class
├── llm_provider/           # LLM API interactions
│   └── client.py           # Ollama/OpenAI client
├── file_metadata_provider/  # File metadata (legacy)
├── cache_provider/         # Query caching
└── chat_history_provider/  # Conversation history
```

4. Class-Level Design

4.1 Core Provider Classes

4.1.1 SkillMetadataProvider

Purpose: SQLite interface for skill definitions and document records

```
SkillMetadataProvider
├── Properties
│   ├── db_path: str                # SQLite database path
│   └── _connection                 # Async connection pool
├── skill_heads Operations
│   ├── create_skill_head()         # Create new skill definition
│   ├── get_skill_head()           # Get by head_id
│   ├── get_skill_head_by_name()   # Get by skill_name
│   ├── list_skill_heads()         # List all heads
│   ├── update_skill_head()        # Modify skill properties
│   ├── delete_skill_head()        # Remove skill definition
│   └── get_documents_for_head()   # Get all docs for a head
├── skill_metadata Operations
│   ├── create_skill()             # Create document record
│   ├── get_skill()               # Get by skill_id
│   ├── list_skills()             # List all documents
│   ├── update_skill()            # Modify document
│   └── delete_skill()            # Remove document
├── Tree Operations
│   ├── get_skill_tree()          # Get hierarchical structure
│   └── migrate_existing_data()   # Migration utility
└── Lifecycle
    ├── initialize_database()      # Create tables
    └── close()                   # Release connections
```

Analogy: Think of `SkillMetadataProvider` as a **library catalog system**:

- `skill_heads` = Book categories (Fiction, Science, History)
- `skill_metadata` = Individual books within each category
- `get_skill_tree()` = "Show me all categories with their books"

4.1.2 VectorStoreProvider

Purpose: FAISS index management for vector similarity search

```
VectorStoreProvider
├── Properties
│   └── backend: str                # "faiss" or "milvus"
```

```

├── _faiss_file_dir          # File embeddings path
├── _faiss_skill_dir        # Skill embeddings path
├── _stores: Dict           # Loaded indices cache
├── Core Operations
│   ├── create_store_from_texts() # Create new index
│   ├── similarity_search()      # Find similar vectors
│   ├── similarity_search_with_score() # With scores
│   └── delete_store()          # Remove index
├── Persistence
│   ├── _save_faiss_store()      # Save to disk
│   ├── _load_faiss_store()     # Load from disk
│   └── _load_all_faiss_stores() # Load all stores
├── Management
│   ├── get_store()             # Get specific index
│   └── list_stores()           # List all indices

```

Analogy: `VectorStoreProvider` is like a **semantic fingerprint database**:

- Each chunk of text gets a unique "fingerprint" (embedding)
- Finding similar content = matching fingerprints
- FAISS = optimized algorithm for finding nearest fingerprints

4.1.3 EmbeddingProvider

Purpose: Text-to-vector conversion using BGE-M3 model

```

EmbeddingProvider
├── Properties
│   ├── model_name: str      # "BAAI/bge-m3"
│   ├── model_kwargs: Dict   # Model configuration
│   └── _model               # Lazy-loaded model instance
├── Core Operations
│   ├── embed_documents()    # Batch text → vectors
│   └── embed_query()        # Single query → vector
├── Internal
│   ├── _lazy_load_model()   # On-demand model loading
│   └── get_underlying_model() # Access raw model

```

Analogy: `EmbeddingProvider` is a **semantic translator**:

- Input: Human-readable text "What is machine learning?"
- Output: 1024 numbers representing meaning
- Similar meanings = similar number patterns

4.2 Core Service Classes

4.2.1 SkillIngestionService

Purpose: Content processing pipeline for skill documents

```
SkillIngestionService
├── Properties
│   ├── chunk_size: int          # 1000 characters
│   ├── chunk_overlap: int       # 200 characters
│   └── text_splitter            # LangChain splitter
├── Processing Pipeline
│   ├── process_skill()          # Full ingestion flow
│   ├── chunk_skill_content()    # Split into chunks
│   └── validate_skill_content() # Content validation
└── Utilities
    ├── generate_skill_id()      # Create unique ID
    ├── generate_unique_skill_id() # Verify uniqueness
    └── estimate_chunk_count()    # Predict chunks
```

4.2.2 SkillRetrievalService

Purpose: Vector search for skill-based queries

```
SkillRetrievalService (extends AbstractRetrievalService)
├── Properties
│   ├── embedding_provider      # For query embedding
│   └── vector_store_provider   # For vector search
├── Core Operations
│   ├── add_content()           # Index new content
│   ├── retrieve_context()      # Search for relevant chunks
│   └── delete_content()        # Remove content
└── Mode
    └── get_retrieval_mode()     # Returns "skill"
```

4.2.3 PromptService

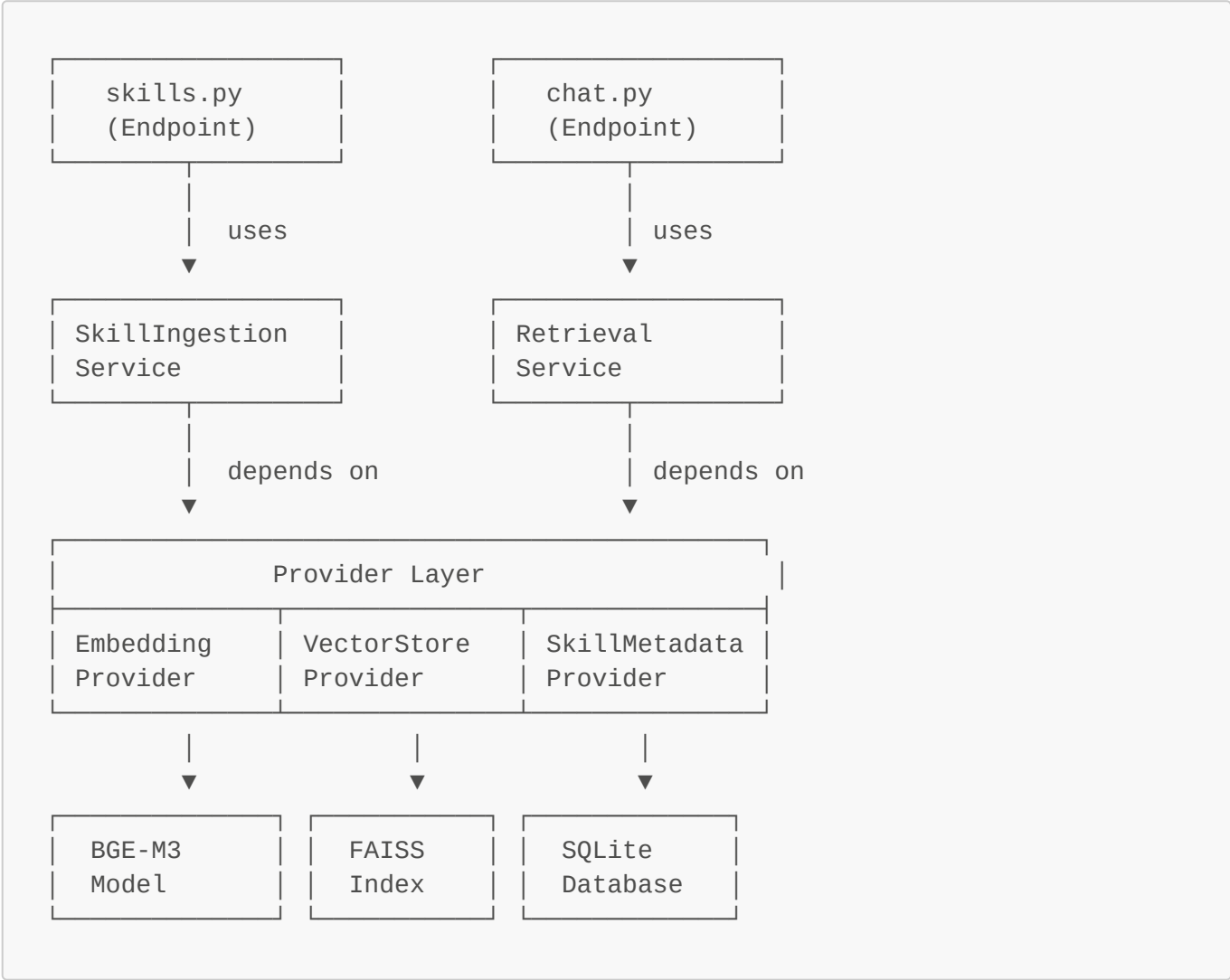
Purpose: RAG prompt assembly for LLM

```
PromptService
├── Templates
│   ├── SYSTEM_PROMPT_TEMPLATE    # Chinese system prompt
│   ├── SYSTEM_PROMPT_TEMPLATE_EN # English system prompt
│   └── MULTI_FILE_SUMMARY_TEMPLATE
└──
```



```
├─ Core Operations
│   ├── build_rag_prompt()          # Full RAG prompt
│   ├── build_simple_prompt()      # Basic prompt
│   └── build_multi_file_summary_prompt()
└─ Utilities
    ├── _assemble_context()        # Format chunks
    └── format_context_for_display()
```

4.3 Class Relationship Diagram



5. Intra-Class Function Analysis

5.1 SkillMetadataProvider Key Functions

create_skill_head() - Lines 721-772

Purpose: Create a new skill definition (category)

```
async def create_skill_head(
    self,
```

```
    skill_name: str,  
    description: str = None,  
    category: str = "General",  
    display_order: int = 0,  
    enabled: bool = True  
    ) -> str:
```

Flow:

1. Generate unique `head_id` using timestamp + UUID
2. Validate `skill_name` is unique
3. INSERT into `skill_heads` table
4. Return `head_id` for document linking

Why needed?: Before this, creating a new skill required manual JSON editing. Now it's automatic.

`get_skill_tree()` - Lines 981-1009

Purpose: Return hierarchical skill structure for UI

Flow:

1. Query all `skill_heads` with LEFT JOIN to `skill_metadata`
2. Group documents under their parent heads
3. Calculate total chunks per head (sum of all documents)
4. Return nested structure

Return structure:

```
[  
  {  
    "head_id": "head_001",  
    "skill_name": "大語言模型大全",  
    "document_count": 3,  
    "total_chunks": 1532,  
    "documents": [  
      {"skill_id": "skill_001", "source_name": "LLM_Book.pdf",  
"chunks": 883},  
      {"skill_id": "skill_002", "source_name": "GPT_Guide.pdf",  
"chunks": 385}  
    ]  
  }  
]
```

`migrate_existing_data()` - Lines 1011-1090

Purpose: One-time migration from old JSON structure

Flow:

1. Find all unique `skill_name` in `skill_metadata`
2. Create corresponding `skill_heads` entries
3. Update `skill_metadata.head_id` foreign keys
4. Return migration statistics

5.2 VectorStoreProvider Key Functions

`create_store_from_texts()` - Lines 218-378

Purpose: Create FAISS index from text chunks

```
async def create_store_from_texts(  
    self,  
    texts: List[str],  
    metadatas: Optional[List[Dict]],  
    store_id: str,  
    store_type: str = "file" # "file" or "skill"  
) -> Dict:
```

Flow:

1. Determine storage directory based on `store_type`
2. Generate embeddings via `EmbeddingProvider`
3. Create FAISS index from embeddings
4. Save index and metadata to disk
5. Cache in memory for fast access

Key insight: `store_type` parameter enables physical isolation between file and skill indices.

`similarity_search_with_score()` - Lines 469-558

Purpose: Find most similar chunks to a query

Flow:

1. Load target FAISS index (by `store_id`)
2. Embed query text
3. Perform vector similarity search
4. Return top-K results with distance scores

Score interpretation:

- Lower score = more similar (L2 distance)
- Typical range: 0.0 (exact match) to 2.0 (very different)

5.3 SkillIngestionService Key Functions

`process_skill()` - Lines 261-360

Purpose: Full ingestion pipeline for skill content

```
async def process_skill(  
    self,  
    skill_name: str,  
    content: str,  
    description: str = None,  
    category: str = "General",  
    head_id: str = None  
) -> Dict:
```

Flow:

1. Validate content (non-empty, reasonable length)
2. Generate unique `skill_id`
3. Chunk content into ~1000 char segments
4. Create FAISS index via `VectorStoreProvider`
5. Store metadata via `SkillMetadataProvider`
6. Return processing statistics

Why 1000 char chunks?: Balance between:

- Too small = loses context, more noise
- Too large = less precise retrieval, token limits

`chunk_skill_content()` - Lines 170-219

Purpose: Split text into overlapping chunks

Algorithm: `RecursiveCharacterTextSplitter`

```
Input: "AAAAAABBBBBBCCCCC..." (long text)  
Output: [  
    "AAAAAA...", (chunk 1: 0-1000)  
    "...AABBBB...", (chunk 2: 800-1800, overlap 200)  
    "...BBCCCC...", (chunk 3: 1600-2600, overlap 200)  
]
```

Why overlap?: Prevents information loss at chunk boundaries.

5.4 PromptService Key Functions

`build_rag_prompt()` - Lines 170-237

Purpose: Assemble complete prompt for LLM

Flow:

1. Select language-appropriate system template
2. Format retrieved chunks as context
3. Combine: system prompt + context + chat history + user query
4. Return message list for LLM

Key design decisions:

- Context placed BEFORE user query (LLM sees context first)
- System prompt includes "don't say 找不到" instruction
- Chat history truncated to last N messages (token management)

6. Data Model Explanation

6.1 Why Two Model Files?

DocAI uses **two distinct model types** for different purposes:

File	Type	Purpose	Use Case
<code>internal.py</code>	TypedDict	Internal data transfer	Provider ↔ Service communication
<code>schemas.py</code>	Pydantic	API validation	Client ↔ Server communication

6.2 app/models/internal.py - TypedDict Models

Why TypedDict?

- **Zero runtime overhead:** No conversion, just type hints
- **Dict compatibility:** Works seamlessly with existing dict-based code
- **IDE support:** Autocomplete and type checking
- **Validation:** Static type checkers (mypy) catch errors

Core Models Explained

ChunkData

```
class ChunkData(TypedDict, total=False):
    content: str          # The actual text chunk
    chunk_index: int      # Position in document (0, 1, 2...)
    metadata: Dict        # Source file, page number, etc.
```

Usage flow: `chunking_strategies` → `input_data_handle_service` → `retrieval_service`

Analogy: Like an index card in a library:

- `content` = the text you wrote on the card
- `chunk_index` = the card number in sequence
- `metadata` = which book it came from, which page

VectorSearchResult

```
class VectorSearchResult(TypedDict):
    content: str          # Matched text chunk
    score: float          # Similarity score (lower = better)
    metadata: Dict        # Chunk metadata
    file_id: Optional[str] # Source document
```

Usage: Return type from `VectorStoreProvider.similarity_search()`

Analogy: Like search results in a search engine:

- `content` = the matching snippet
- `score` = relevance ranking
- `metadata` = additional context

QueryExpansionResult

```
class QueryExpansionResult(TypedDict):
    original_query: str          # User's original question
    intent: str                 # Detected intent type
    expanded_questions: List     # Generated sub-questions
    cache_hit: bool             # Was this cached?
```

Usage: Query enhancement service output

Example:

```
{
    "original_query": "What is RAG?",
    "intent": "definition_seeking",
    "expanded_questions": [
        "What does RAG stand for?",
        "How does RAG work technically?",
        "What are benefits of RAG?"
    ],
    "cache_hit": False
}
```

PromptContext & PromptMessages

```
class PromptContext(TypedDict):
    system_prompt: str          # Instructions for LLM
    user_query: str             # User's question
    context_chunks: List[str]   # Retrieved relevant chunks
```

```
chat_history: List          # Previous messages

class PromptMessages(TypedDict):
    messages: List[LLMMessage] # Final LLM input format
    total_tokens_estimate: int  # Estimated token count
```

Usage: Intermediate stages of prompt construction

6.3 app/models/schemas.py - Pydantic Models

Why Pydantic?

- **Runtime validation:** Invalid data throws clear errors
- **Auto-documentation:** OpenAPI/Swagger schema generation
- **Serialization:** JSON conversion built-in
- **Field constraints:** min/max length, patterns, etc.

Core Models Explained

ChatRequest

```
class ChatRequest(BaseModel):
    query: str = Field(..., min_length=1, max_length=2000)
    file_ids: List[str] = Field(..., min_items=1)
    session_id: Optional[str] = None
    enable_query_enhancement: bool = True
    top_k: int = Field(default=5, ge=1, le=20)
    temperature: float = Field(default=0.7, ge=0.0, le=2.0)
```

Validation rules:

- **query:** 1-2000 characters (prevents empty/huge queries)
- **file_ids:** At least one file required
- **top_k:** Between 1-20 (reasonable retrieval count)
- **temperature:** 0.0-2.0 (LLM creativity range)

ChatResponse

```
class ChatResponse(BaseModel):
    answer: str          # LLM's response
    session_id: str      # Conversation ID
    query_enhancement: Dict # Expansion details
    retrieved_chunks: List[Dict] # Source chunks
    metadata: Dict       # Timing, tokens, etc.
```

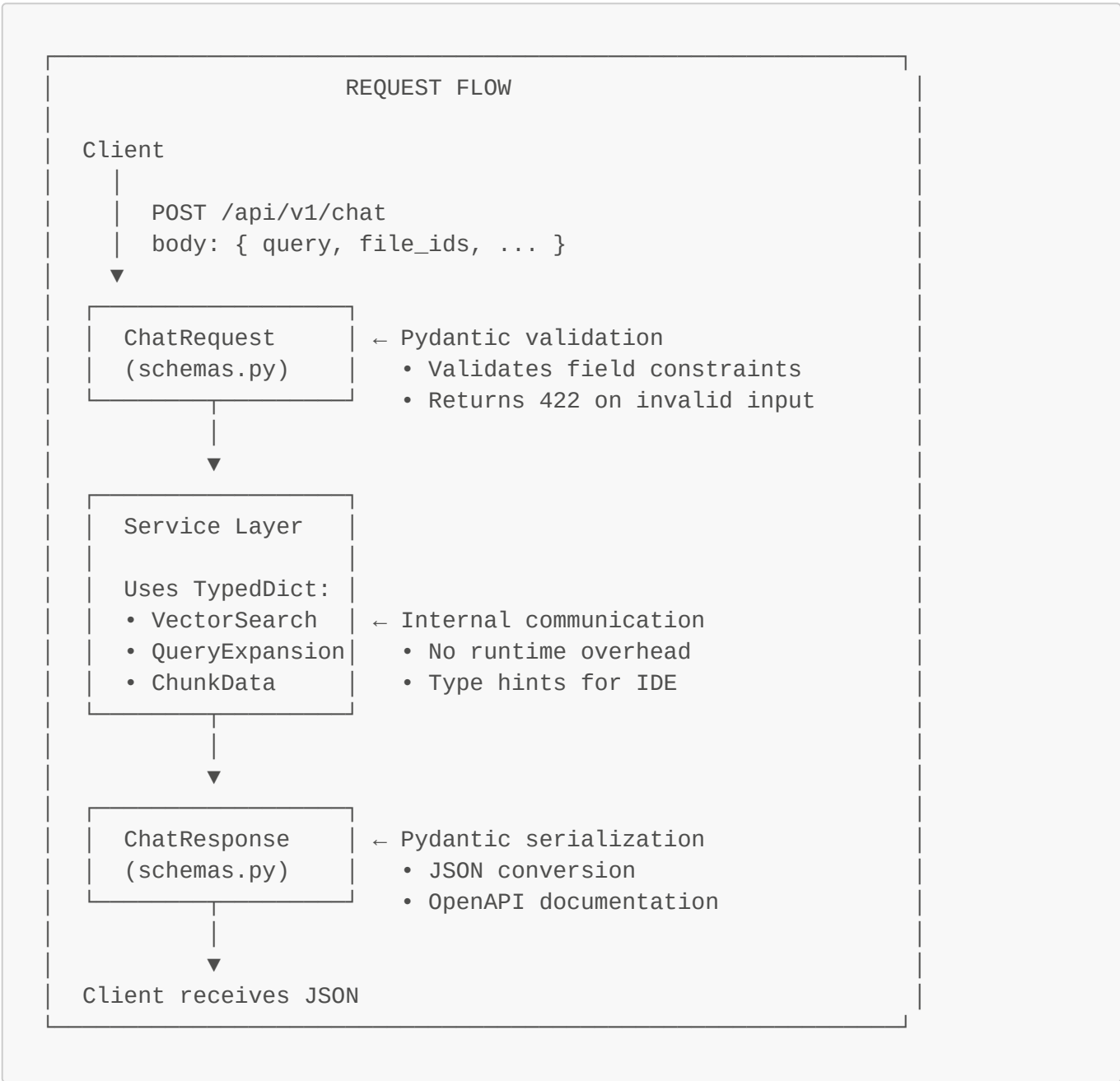
Purpose: Standardized response format for all chat endpoints

UploadResponse

```
class UploadResponse(BaseModel):
    file_id: str
    filename: str
    file_size: int
    chunk_count: int
    embedding_status: str # "pending", "completed", "failed"
    message: str
```

Purpose: Feedback after file upload

6.4 Model Usage Throughout System



6.5 Design Philosophy

Separation of concerns:

- `schemas.py` = External contracts (what clients see)
- `internal.py` = Internal contracts (what services see)

Trade-offs:

Aspect	TypedDict	Pydantic
Runtime overhead	None	Some (validation)
Runtime validation	No	Yes
Serialization	Manual	Automatic
IDE support	Good	Excellent
Error messages	Type checker	Runtime

When to use which?

- **TypedDict:** High-throughput internal data (thousands of chunks)
- **Pydantic:** API boundaries (request/response validation)

7. Summary

7.1 Architecture Highlights

1. **Dual RAG System:** File-based (Milvus) and Skill-based (FAISS) operate independently
2. **Layered Architecture:** Endpoints → Services → Providers → Storage
3. **Single Source of Truth:** `skill_heads` table replaced JSON config
4. **Physical Isolation:** Separate directories for file vs skill indices

7.2 Key Design Patterns

Pattern	Implementation	Benefit
Repository	Providers (VectorStore, Metadata)	Database abstraction
Strategy	AbstractRetrievalService	Swappable retrieval modes
Factory	<code>get_*_provider()</code> functions	Singleton management
TypedDict vs Pydantic	Internal vs External	Performance vs Validation

7.3 Data Flow Summary

Upload: PDF → Extract → Chunk → Embed → FAISS + SQLite

Query: Question → Embed → Search → Context → LLM → Answer

7.4 Technology Rationale

Choice	Why
FAISS over Milvus (Skills)	Lightweight, no server needed
SQLite over MySQL (Skills)	Local, file-based simplicity
BGE-M3	Multilingual, 1024-dim balance
TypedDict	Zero overhead for internal data
Pydantic	Validation at API boundaries

Report generated: 2025-12-04 DocAI Version: skill_based_docai_dev Architecture: skill_heads Single Source of Truth